(27-1-26)

Data

It refers to raw facts, figures or values.

Data types

A data type defines the kind of data a variable can hold and operations that can be performed on it.
what values to assign and what operations can be performed
There are two types of data types

Built in data types

These are primitive data types that are directly supported by the programming language.

Example

int - integer values
float - Decimal numbers
char - single character
bool - Boolean values (true/false)
string- Sequence of some characters

Derived data types

Derived data types are constructed using one or more built-in types and operations.

Example

Array, List, Stack, Queue, Trees, Graphs

Data Structures

A Data structure is a way to organize, store, and manage data so that it can be used efficiently in a computer program.

A Data Structure defines:
How data is stored in memory
How you can access or manipulate it.

Basic operations in Data Structures

Different data structures support various operations. Common operations are

- Traversal - Accessing each element o f the data structure
- Searching - Finding a specific element
- Insertion - Adding a new element
- Deletion - Removing an existing element
- Sorting - Arranging elements in a particular order
- Merging - Combining two or more data structures

Types of Data Structures

Data Structures can be broadly categorized based on how data is organized, stored, and accessed.

1. Linear Data Structure
2. Non-Linear Data Structure

Linear Data Structure

In a linear data structure, elements are arranged sequentially or linearly, where each element is connected to its previous and next element. These structures are simple to implement and are often used when data needs to be processed in order.
(Continuous memory allocation for the elements, if there is space available, continuously.)

Example

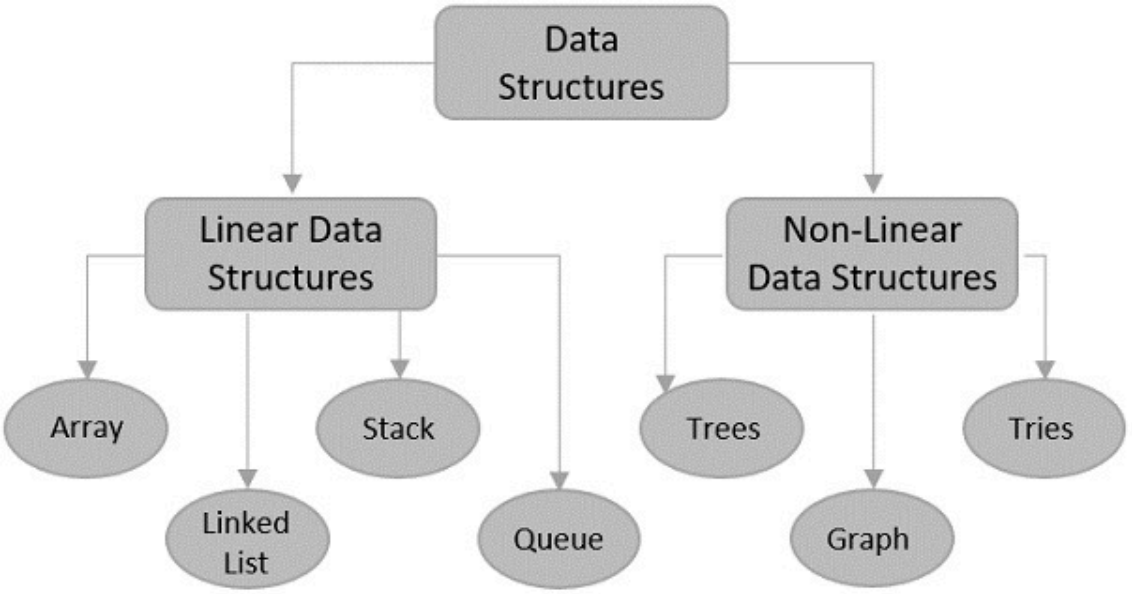
Array - Fixed size collection of similar data types.
Stack - Follows Last-in-First-out (LIFO) principle.
Queue - Follows First-in-first-out.
Linked List - A series of nodes where each node contains data and a pointer to the next node.

Non-Linear Data Structure

In non-linear data structure elements are not arranged in a sequential manner. Instead, they follow a hierarchical or interconnected structure, allowing for more complex relationship between data elements.

Example

Tree - A hierarchical structure consisting of nodes, with a single root node and sub-nodes (children).
Graph - A set of nodes connected by edges, used to represent networks.



Static data structures

Data structures can be categorized based on how memory is allocated at compile-time and remains fixed throughout the program's lifecycle. This means the size of the data structure must be known in advance.

Example

Array,

Dynamic

Dynamic data structures allow memory allocation to occur at runtime.

Example

Linked Lists, stacks and queues.

When solving a problem using computer program, there are often multiple algorithms that can achieve the same goal.

Algorithm analysis

It is the process of evaluating the performance of an algorithm, typically

---

(28-1-26) Day 2

Searching Techniques

Searching is a fundamental operation in computer science that involves finding the presence or location of an element within the data structure , such as an array or list.

The techniques differ in their approach, time complexity, and stability based on whether the data is sorted or unsorted.

1. **Linear Search** : A simple method that checks each element sequentially until the target is found or the list ends
2. Binary search : A fast, divide-and-conquer approach that requires sorted data and repeatedly divides the search interval in half.
3. Uniform Binary Search : A variation of binary search that uses a precomputed table to reduce computations.
4. Interpolation search : An improved variant
5. Fibonacci search :

Liner Search

Liner Search is the simplest searching algorithm that checks every element in a list sequentially until it finds the target element or reaches the end of the list. It does not require the data to be sorted and works well on small or unsorted datasets.

Algorithm: Liner Search

Problem: Search for a given element (key) in an array using the linear search technique.
Input: An array arr [ ] of size n. and a target element key.
Output: Index of the key if found: otherwise -1

Algorithm Steps

1. start from the first element of the array
2. compare the current element with the target element key.
3. if the current element matches key, return the index
4. if not, move to the next element
5. repeat steps 2-4 until the end of the array is reached
6. if no match is found after traversing the entire array, return -1

Pseudo code: Linear Search

1. function LINEARSEARCH(arr, key)
2. for i <-- 0 to length(arr) - 1
3.
```
  if arr[ i ] = key then
```

```
    return i
```

```
  end if
```

end for
return -1
end function

Time Complexity

- Best case: O(1) - target is found at the first position.
- Worst case: O(n) - target is not present or at the last position.
- Average case: O(n)

Space complexity

Not much additional space it taken.

```
public class LinearSearch {
    public static int linearsearch(int[] arr, int key){
        for(int i = 0; i < arr.lenght; i++){
            if(arr[i] == key){
                return i;
            }
        }return -1;
    }
    public static void main(String[] args){
        int[] arr = {10, 15, 12, 25, 30};
        int key = 30;
        int result = linersearch(arr, key);
        if (result != -1){
            System.out.println("The element is found at index " + result);
        }else{
            System.out.println("The element is not found");
        }
    }
}
```

Binary Search

Binary search is an efficient searching algorithm that works on sorted arrays. It follows the divide and conquer approach by repeatedly dividing the search interval in half. If the value of the search key is less than the middle element of the interval, the search continues on the left subarray. Otherwise, it continues on the right subarray. This process repeats until the key is found or the interval becomes empty.

Key characteristics

- Requires the array to be sorted
- Time complexity: O(logn)
- Space Complexity: O(1) for iterative implementation

Code

```
public class BinarySearch {
public static int binarySearch(int[] arr, int key){
int low = 0, high = arr.lenght-1;

while (low <= high){
 int mid = (low + high)/2;

 if (arr[mid] == key)
   return mid;
 else if (key < arr[mid])
   high = mid-1;
 else
   low = mid+1;

}
return -1;
}

public static void main(String[] args){
int[] sortedArray = {5, 10, 15, 20, 25, 30, 35};
int key = 25;

int result = binarySearch(sortedArray, key)

 if(result != -1)
   System.out.println("Element is found at index: "+ result);
 else
   System.out.println("element not found");
}
}
```

Asymptotic Analysis

As input sizes grow very large, it becomes impractical to measure an algorithm's exact running time in seconds or steps. Instead, we use asymptotic analysis to describe the general trend of an algorithm's performance as the input size approaches infinity.

Asymptotic analysis focuses on the dominant factors that influence growth while ignoring constant multipliers and lower-order terms that have little effect on large inputs.

This approach allows us to:

1. Abstract away machine-specific details and implementations differences.
2. Compare algorithms based on their scalability and efficiency.
3. Understand the upper and lower bounds .

Types of Asymptotic Notations

Asymptotic notations provide formal ways to describe the growth rates of functions representing an algorithm's time or space complexity. The five main asymptotic notations are:

1. Big O notations (O) - Provides an upper bound on the growth rate. It describes the worst-case scenario, meaning the algorithm's running time will not exceed this bound for sufficiently large inputs
2. Big Omega Notation (Ω) - Provides a lower bound on the growth rate. It describes the best-case scenario, meaning the running time will be at least this much
3. Big theta (**Θ**)
4. Small (o)
5. Small Omega (**ω**)

Space complexity

Space Complexity refers to the amount of memory or space an algorithm needs to run to completion.

Types of Memory used

1. Instruction Space:
2. Environmental Stack:
3. Heap:
4. Input/output data space:

Common Space Complexities

1. O(1): Constant space - no matter the input size, the algorithm uses a fixed amount of memory.
2. O(n): Linear space - memory used grows proportionally with input size.
3. O(n2): Quadratic space - used in some matrix or graph algorithms

---

(29-1-26)

Recursion

Recursion is a programming technique in which a function calls itself directly or indirectly to solve a smaller instance of a problem until a base condition is met. It is a powerful tool used for solving problems that can be broken down into similar sub-problems.

Key components of recursion

Base case: The condition under which the recursion stops.
Recursive case: The part of the function where it calls itself with a smaller or simpler input.

Example: Factorial of a Number

```
public class factorial{
static int
}
```

Types of recursion

1. Direct Recursion

Direct Recursion: A Function calls itself directly.

Example

```
public class DirectRecursion{
static void countDown(int n){
 if (n == 0) return;
 System.out.println(n);
 countDown(n - 1);
}
public static void main(String[] args){
countDown(5)
}
}
```

2. Indirect Recursion

Indirect Recursion: A function that calls another function that eventually calls the original function.

```
public class IndirectRecursion {
static void functionA(int n){
  if (n <= 0) return;
  System.out.println("A: " + n);
  functionB(n - 1);
}
static void functionB(int n){
  if (n <= 0) return;
```

```
  System.out.println("B: " + n);
  functionA(n / 2);
}

public static void main(String[] args){
functionA(10);
}
}
```

3. Tail Recursion

Tail Recursion: The recursive call is the last statement in the function.

```
public class TailRecursion {
 static void printNumbers(int n, int k){
   if (n == 0) return;
   System.out.println(k);
   printNumbers(n - 1, k + 1);
 }
 public ststic void main(Stering[] args){
   printNumbers(5, 1);
 }
}
```

Uniform Binary Search

Uniform Binary Search is a variation of the classic Binary Search Algorithm. While Binary search recalculates the midpoints during every iteration, Uniform Binary Search precomputes and stores the midpoints for a fixed-size array.

Key Characteristics

1. Precomputed midpoints are used instead of calculating them dynamically
2. Useful for systems with fixed-size memory and limited processing power
3. Same time complexity as binary search: O(logn).

Example

Consider a sorted array and a key to be searched:

```
A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
```

Key to be searched: Key = 23
Number of elements: n = 10

Precomputation of jumps

The jump sizes are chosen as powers of 2 less than equal to n:
Jumps = {8,4,2,1,}
The initial position is set to the largest jump value:
pos = 8

step	Jump size	position (pos)		Comparision	Action
1	8	8	72	72 > 23	Move left: pos = 8 - 4 = 4
2	4	4	16	16 < 23	Move Right: pos = 4 + 2 = 6
3	2	6	38	38 > 23	Move Left: pos = 6 - 1 = 5
4	1	5	23	23 = 23	Key found

```
public class UniformBinarySearch {
  static final int MAX_SIZE = 10;
  static int[] lookupTable = new int[MAX_SIZE];

  static void createLookupTable(int n) {
    int pow = 1;
    int index = 0;

    do {
       pow <<= 1;
       lookupTable[index] = (n + (pow >> 1)) / pow;
    } while (lookupTable[index++] != 0);
  }
  static int uniformBinarySearch(int[] arr, int target) {
     int index = lookupTable[0] - 1;
     int step = 0;

     while (lookupTable[step] != 0){

        if (target == arr[index]) {
        return index;
        }
        else if (target < arr[index]){
        index -= lookupTable[++step];
        }
        else {
         index += lookupTable[++step];
        }
        if (index < 0 || index >= arr.length){
        return -1;
        }
     }
```

```
      return -1;
   }
   public static void main(String[] args){
     int[] arr = {1, 3, 5, 6, 7, 8, 9};
     int target = 3;

     createLookupTable(arr.length);

     int result = uniformBinarySearch(arr, target);

     if (result != -1)
     System.out.println("Element " + target + " found at index = "+ result);
     else
     System.out.println("Element "+ target + "  not foumd in  the array.");
   }
 }
```

Iteration	pow	pow >> 1		lookup_table
1	2	1	(7+1)/2 = 4	4
2	4	2	(7+2)/4 = 2.25	2
3	8	4	(7+4)/8 = 1.375	1
4	16	8	(7+8)/16 = 0.94	0

Interpolation Search

Interpolation Search is an improved variant of binary search, best suited for uniformly disturbed sorted arrays. Unlike Binary search which always checks the middle element, Interpolation search estimates the position of the desired key based on the value being searched.

It works on the principle of **proportional division** the closer the value is to the end or beginning, the closer the guess is to that side.

Key Idea

1. Works only on sorted arrays.
2. Improves performance by using the value of the element being searched.
3. Ideal for uniformly distributed values.

Formula

The position of the element is calculated using the formula:

low & high are search boundaries
A[ ] is sorted array

Consider A =

```
public class InterpolationSearch {
 public static int interpo;ationSearch(int[] arr, int key) {
   int low = 0;
   int high = arr.length - 1;

   while (low <= high && key >= arr[low] && key <= arr[high]){
     int pos = low + ((key - arr[low]) * (high - low)) / (arr[high] -
arr[low]);

     if (arr[pos] == key){
       return pos;
     }

    if (arr[pos] < key){
      low = pos + 1;
    }
    else {
       high = pos -1;
    }
   }
   return -1;
 }
 public static void main(String] args){
    int[] arr = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
    int key = 70;

    int result = interpolationSearch(arr, key);

    if (result != -1) {
        System.out.println("Element " + key + " found at index" + result);
    } else {
        System.out.println("Element " + key + " not found ");
    }
 }
}
```

Fibonacci Search

Fibonacci Search is an efficient searching algorithm that uses Fibonacci numbers to divide the array and reduce the search space. It is particularly useful for sorted arrays and can perform better than binary search in some cases, especially when the cost of accessing the middle element is high.

Key Idea

1. Uses Fibonacci numbers to determine the index to be checked.

2. Divides the array into sections whose sizes are consecutive Fibonacci numbers.
3. Works well with large sorted arrays.

Algorithm

1. Find Fibonacci number F(m) such that F(m) >= n, where n is the size of the array.
2. Compare the key with the elements at index min(offset + F(m-2), n-1).
3. Depending on the comparison:
   - If key < element, move to left subarray and reduce Fibonacci numbers.
   - if key > element, move to right subarray and reduce Fibonacci numbers.
4. Repeat until the element is found or search space is exhausted.

Example

Consider the sorted array:

we want to search for:
key = 85
array size: n = 11 Smallest Fibonacci number >= F(7) = 13

Dry Run

Iteration	fibM	fibMMm1	fibMMm2	offset	i	A[i]	Comparison/ Action

---

(30-1-26)

Sorting Technique

Sorting is a fundamental operation in computer science that arranges elements of a dataset in a specific order, usually ascending or descending. Sorted data allows efficient searching, merging and analysis, and is a prerequisite for many algorithms.

There are many sorting techniques, each with specific strengths and weaknesses. The choice of a sorting algorithm depends on factors such as:

1. Size of memory
2. Memory constraints
3. Nature of the data (random, partially sorted, or nearly sorted)

Key Terms in Sorting

1. Stable sort: A sorting algorithm is stable is it preserves the ***relative order of equal elements***
   Example: If two students have the same marks, a stable sort keeps their original order.
   Example of stable sorts: Bubble sort, Merge sort, Insertion sort.
2. In-place sort: A sorting algorithm is in-place if it sorts the array ***without using extra memory.***
   Example: Quick sort, Selection sort, Bubble sort
3. Adaptive sort: An adaptive sorting algorithm takes advantage of ***existing order in the input data***. The more sorted the array already is, the fewer operations it performs.
   Examples: Insertion sort, Timsort, Optimized Bubble Sort
4. Comparison-Based Sort: A sorting algorithm is comparison-based if it determines the order of elements by ***comparing pairs of elements***.
   Example: Bubble Sort, Merge Sort, Quick Sort, Heap Sort.
5. Non-Comparison Sort: A sorting algorithm that does not compare elements directly but uses the element values themselves to sort.
   Example: Counting Sort, Radix Sort, Bucket Sort.

Term	Meaning	Example
Stable	Maintains relative order of equal elements	Merge Sort, Bubble Sort
In-Place	Sorts using constant extra space	Quick Sort, Selection Sort
Adaptive	Performs better is input is partially sorted	Insertion Sort
Comparison-Based	Determines order using comparisions	Bubble sort, Heap sort
Non-Comparison	Sorts using element values	Counting sort, Radix Sort

---

Bubble Sort

Example

Let a array be (66, 55, 44, 33, 22, 11)
Pass 1
66 55 44 33 22 11
55 66 44 33 22 11
55 44 66 33 22 11
55 44 33 66 22 11
55 44 33 22 66 11
55 44 33 22 11 66

Pass 2
55 44 33 22 11 66
44 55 33 22 11 66
44 33 55 22 11 66
44 33 22 55 11 66
44 33 22 11 55 66

Pass 3
44 33 22 11 55 66
33 44 22 11 55 66
33 22 44 11 55 66
33 22 11 44 55 66

Pass 4
33 22 11 44 55 66
22 33 11 44 55 66
22 11 33 44 55 66

Pass 5
22 11 33 44 55 66
11 22 33 44 55 66

Bubble Sort is one of the simplest sorting algorithms. It works by repeatedly swapping adjacent elements if they are in the wrong order. This process is repeated until the list is sorted.

Algorithm Description

Given an array of n elements, the algorithm performs the following steps:

1. Compare each pair of the adjacent elements.
2. If the current element is greater than the next element, swap them.
3. After each pass, the largest element among the unsorted elements "bubble up" to its correct position at the end of the array.
4. Repeat the above steps for the remaining unsorted part of the array.

Time Complexity

1. Best Case: O(n) - when the array is already sorted
2. Average Case: O(n^2)
3. Worst Case: O(n^2)

```
public class BubbleSort{

    public static void bubbleSort(int[] arr){
        int n = arr.lenght;
```

```
        for (int i = 0; i < n - 1; i++ ) {
            for(int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]){
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
    public static void printArray(int[] arr){
        for (int num : arr) {
            System.out.print(num + " ");
        }
        System.out.println();
    }
    public static void main(String[] args) {
         int[] arr = {5, 3, 8, 4, 2};

         System.out.println("Original array:");
         printArray(arr);

         bubbleSort(arr);
         System.out.println("Sorted array:");
         printArray(arr);
    }
}
```

Insertion Sort

Insertion Sort is a simple and intuitive sorting algorithm that builds the sorted array one element at a time. It works by repeatedly taking the next element and inserting it into the correct position in the already sorted part of the array.

Algorithm Description

Given any array of n elements, the algorithm follows these steps:

1. Start from the second element (index 1), as the first element is considered sorted.
2. Compare the current element with the elements in the sorted portion (to the left).
3. Shift all the elements in the sorted portion that are greater than the current element to the right by one position.
4. Insert the current element into the correct position.

Example

Initially	23, 1, 10, 5, 2	23, 1, 10, 5, 2
First Pass	23, 1, 10, 5, 2	1, 23, 10, 5, 2
Second Pass	1, 23, 10, 5, 2	1, 10, 23, 5, 2
Third Pass	1, 10, 23, 5, 2	1, 5, 10, 23, 2
Fourth Pass	1, 5, 10, 23, 2	1, 2, 5, 10, 23

Time Complexity

Best case: O(n)
Average case: O(n^2)
Worst Case: O(n^2)

```
public class InsertionSort {
    public static void insertionSort(int[] arr){
        for (int i = 1; i < arr.length; i++){
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }
    public static void printArray(int[] arr){
        for (int num : arr) System.out.print(num + " ");
        System.out.println();
    }
    public static void main(String[] arr){
        int[] arr = {};

        System.out.println("Elements before sorting:");
        printArray(arr);
        insertionSort(arr);
        System.out.println("The elements after sorting");
        printArray(arr);
    }
}
```

Selection Sort

Selection Sort is a simple comparison-based sorting algorithm. It works by repeatedly finding the minimum element from the unsorted part of the array and swapping it with the first

element of the unsorted part.

Algorithm Description

Given an array of n elements, the steps are:

1. Start from the first element and assume it as the minimum.
2. Search the rest of the array to find the actual minimum element.
3. Swap the minimum element found with the first element.
4. Move the boundary of the sorted part one position forward.
5. Repeat the above steps for the remaining unsorted portion until the entire array is sorted.

Time Complexity

Best case: O(n^2)
Average case: O(n^2)
Worst case: O(n^2)

```
public class SelectionSort {
    public static void selectionSort(int[] arr) {
        int n = arr.lenght;
        for(int i = 0; i < n - 1; i++) {
            int minIndex = i;
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
    public static void printArray(int[] arr) {
        for (int num : arr) System.out.print(num + " ");
        System.out.println();
    }
    public static void main(String[] args){
        int[] arr = {5, 3, 8, 4, 2};

        System.out.println("The elements before sorting");
        printArray(arr);
        selectionSort(arr);
        System.out.println("The elements after sorting");
        printArray(arr);
    }
}
```

(31-1-26)

Quick Sort

Example

0	1	2	3	4	5	6	7	8	9	10	11	1
38	08	16	06	79	57	24	56	02	58	04	70	4
Pivot	08	16	06	Up	57	24	56	02	58	Dn	70	4
Pivot	08	16	06	04	57	24	56	02	58	79	70	4
Pivot	08	16	06	04	Up	24	56	Dn	58	79	70	4
Pivot	08	16	06	04	02	24	56	57	58	79	70	4
Pivot	08	16	06	04	02	Dn	Up	57	58	79	70	4
24	08	16	06	04	02	38	56	57	58	79	70	4
pivot	08	16	06	04	Dn	Up	56	57	58	79	70	4

(02 08 16 06 04 24) 38 (56 57 58 79 70 45)

Merge Sort

Example

Radix Sort

Example

Array: 170, 45, 75, 90, 802, 24, 2, 66

Pass 1 Units Digit (exp = 1)
Buckets (0-9)
0 - 170, 90
2 - 802, 2
4 - 24
5 - 45, 75
6 - 66
Array after Pass 1 :
170, 90, 802, 2, 24, 45, 75, 66

Pass 2 Tens digit (exp = 10)
Buckets:

0 - 802, 2
2 - 24
4 - 45
6 - 66
7 - 75, 170
9 - 90
Array after Pass 2: 802, 2, 24, 45, 66, 170, 75, 90

(27-2-26)

Heap Sort

Heap sort is a comparison-based sorting technique based on Binary Heap data structure. It is similar to the selection sort where we first find the minimum element and place the minimum element at the beginning. Repeat the same process for the remaining elements.

```
class MaxHeap {
    int[] heap;
    int size;
    int capacity;

    MaxHeap(int capacity) {
        this.capacity = capacity;
        heap = new int[capacity];
        size = 0;
    }

    // Insert element
    void insert(int value) {
        if (size == capacity) {
            System.out.println("Heap is full");
            return;
        }

        heap[size] = value;
        int i = size;
        size++;

        // Sift Up
        while (i > 0 && heap[(i - 1) / 2] < heap[i]) {
            int temp = heap[i];
            heap[i] = heap[(i - 1) / 2];
            heap[(i - 1) / 2] = temp;
            i = (i - 1) / 2;
        }
    }
```

```
    // Remove maximum element
    int extractMax() {
        if (size <= 0) return -1;

        int root = heap[0];
        heap[0] = heap[size - 1];
        size--;

        heapify(0);

        return root;
    }

    // Heapify method
    void heapify(int i) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < size && heap[left] > heap[largest])
            largest = left;

        if (right < size && heap[right] > heap[largest])
            largest = right;

        if (largest != i) {
            int temp = heap[i];
            heap[i] = heap[largest];
            heap[largest] = temp;

            heapify(largest);
        }
    }
}
```

```
void heapify(int[] arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
```

```
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}
```

```
for (int i = n/2 - 1; i >= 0; i--)
    heapify(arr, n, i);
```

```
class HeapSort {

    void heapify(int[] arr, int n, int i) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest])
            largest = left;

        if (right < n && arr[right] > arr[largest])
            largest = right;

        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;

            heapify(arr, n, largest);
        }
    }

    void sort(int[] arr) {
        int n = arr.length;

        // Step 1: Build Max Heap
        for (int i = n / 2 - 1; i >= 0; i--)
            heapify(arr, n, i);

        // Step 2: Extract elements
        for (int i = n - 1; i > 0; i--) {
            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;

            heapify(arr, i, 0);
        }
```

```
    }

    public static void main(String[] args) {
        int[] arr = {4, 10, 3, 5, 1};

        HeapSort hs = new HeapSort();
        hs.sort(arr);

        System.out.println("Sorted array:");
        for (int num : arr)
            System.out.print(num + " ");
    }
}
```

```
import java.util.Scanner;

public class HeapSort {

    // Heapify function
    void heapify(int[] arr, int n, int i) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest])
            largest = left;

        if (right < n && arr[right] > arr[largest])
            largest = right;

        if (largest != i) {
            int temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;

            heapify(arr, n, largest);
        }
    }

    // Heap Sort function
    void sort(int[] arr) {
        int n = arr.length;

        // Build max heap
        for (int i = n / 2 - 1; i >= 0; i--)
            heapify(arr, n, i);

        // Extract elements
        for (int i = n - 1; i > 0; i--) {
```

```
            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;

            heapify(arr, i, 0);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        HeapSort hs = new HeapSort();
        hs.sort(arr);

        System.out.println("Sorted array:");
        for (int num : arr) {
            System.out.print(num + " ");
        }

        sc.close();
    }
}
```

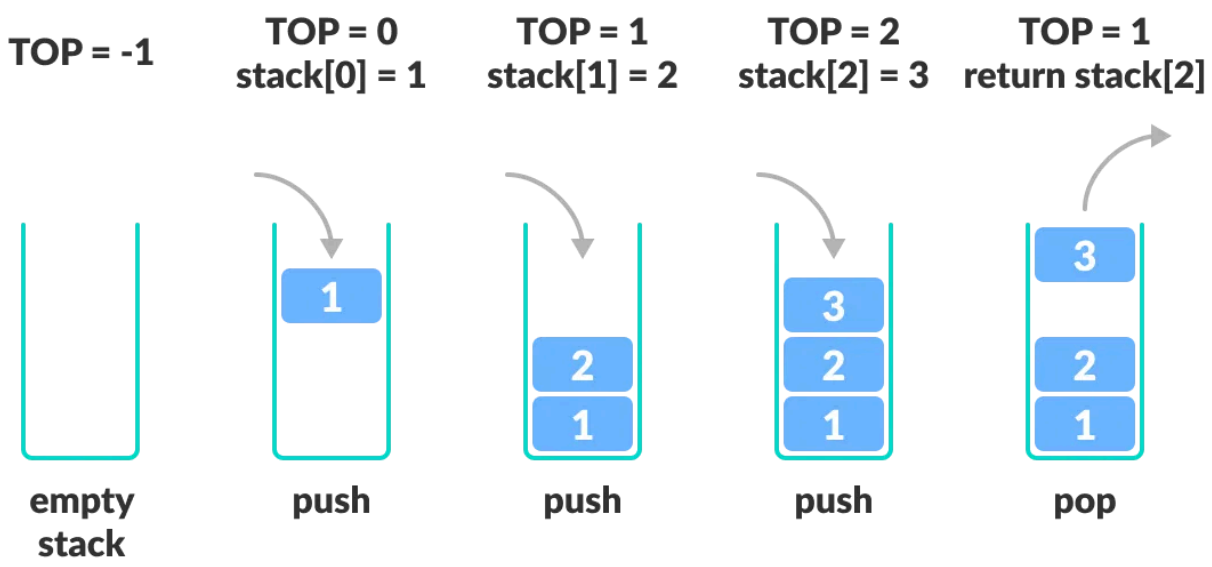
(2-2-26)

Stack Data Structure

Stacks is a linear data structure that follows the Last In First Out (LIFO) principle. The element inserted last is the one that is removed first.

Characteristics of Stack

LIFO Order: The last inserted element is the first one to be removed.
Access Restriction: Only one end, called the top, is accessible for insertion and deletion.
Dynamic/ Static Implementation: Can be implemented using arrays (static) or linked lists (dynamic).



Basic Operations

Push: Add an element to the top of the stack
Pop: Remove and return the top element of the stack
Peak/Top: Return the top element without removing it.
IsEmpty: Check is the stack is empty.
IsFull: (For array implementation) Check if the stack is full.

Applications of Stack

1. Reversing a sting
2. Expression evaluation and conversion (infix to postfix/prefix)
3. Backtracking algorithms (e.g., maze or puzzle solving)
4. Undo mechanisms in editors
5. Syntax parsing in compilers

Time Complexity

Push: O(1)
Pop: O(1)
Peek: O(1)

```
import java.util.Scanner;

public class StackUsingArray {
    static int MAX_SIZE;
    static int[] stack;
    static int top = -1;

    static void push(int value) {
        if (isFull()) {
```

```
            System.out.println("stack overflow! cannot push");
            return;
        }
        stack[++top] = value;
        System.out.println(value + "pushed to stack.");
    }

    static void pop() {
        if (isEmpty()) {
            System.out.println("Stack is Underflow! cannot pop");
            return;
        }
        System.out.println(stack[top--] + " popped from stack.");
    }

    static void peak() {
        if (isEmpty()) {
            System.out.println("Stack is empty!");
            return;
        }
        System.out.println("Top element is: " + stack[top]);
    }

    static boolean isEmpty() {
        return top == -1;
    }

    static boolean isFull() {
        return top == MAX_SIZE - 1;
    }

    static void display() {
        if (isEmpty()) {
            System.out.println("Stack is empty.");
            return;
        }
        System.out.println("stack elements from top to bottom: ");
        for (int i = top; i >= 0; i--){
            System.out.println(stack[i]);
        }
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter stack size: ");
        MAX_SIZE = scanner.nextInt();
        stack = new int[MAX_SIZE];

        int choice;
        do {
```

```
        System.out.println("\n--- Stack Operations Menu ---");
        System.out.println("1. push");
        System.out.println("2. pop");
        System.out.println("3. peak");
        System.out.println("4. display");
        System.out.println("5. check if empty");
        System.out.println("6. check if full");
        System.out.println("0. exit");
        System.out.println("Enter your choice: ");
        choice = scanner.nextInt();

        Switch (choice) {
           case 1:
               System.out.println("Enter value to push: ");
               int val = scanner.nextInt();
               push(val);
               break;
           case 2:
               pop();
               break;
           case 3:
               peek();
               break;
           case 4:
               display();
               break;
           case 5:
               System.out.println(isEmpty() ? "Stack is empty. " : "Stack
is not empty");
               break;
           case 6:
               System.out.println(isFull() ? "Satck is full" : "Stack is
not full.");
               break;
           case 0:
               System.out.println("Exiting program....");
           break;
           default:
               System.out.println("Invalid choice! Try again");
        }
       } while (choice != 0);
       scanner.close();
    }
}
```

(31-3-26)

Expression Evolutions

- Arithmetic Expressions can be written in one of three forms
- Infix expression: Operands are written between the operands they operate on
- Example: 3+4
- Prefix expression: Operators are written before the operands
- Postfix expression: Operators are written after the operands

Operator Precedence

It determines which operation is performed first in expression with more than one operators with different precedence.

Precedence rules:

1. ( ), [ ], { }
2. ^
3.
    - , /
4.
    - , -

Associativity Rule

Operators Associativity is used when two operators of same precedence appears in an expression.

- ^ : Right to Left
-
    - , / : Left to Right
-
    - , - : Left to Right

Infix to Prefix

While Encountering brackets pop every thing between closing brackets ")" and opening brackets "("

Without Stack

Infix: a + b * c - d / e ^ f

- a + b * c - d / ^ e f
- a + * b c - d / ^ e f
- a + * b c - / d ^ e f

- a *b c - / d ^ e f*
  Answer : -+ a b c / d ^ e f

With Stacks

Question : a + b *c - d / e ^ f*
Reverse expression : f ^ e / d - c b + a

Infix	Stack	Prefix
f		f
^	^	f
e	^	fe
/	/	fe^
d	/	fe^d
-	-	fe^d/
c	-	fe^d/c
*	-*	fe^d/c
b	-*	fe^d/cb
+	-+	fe^d/cb*
a	-+	fe^d/cb*a
	-	f e ^ d / c b * a +
		f e ^ d / c b * a + -

Reverse : - + a * b c / d ^ e f

K + L - M *N + (O ^ P)* W / U / V * T + Q

Reverse Expression : Q + T *V / U / W*) P ^ Q (+ N * M - L + K

INFIX	STACK	PREFIX
Q		Q
+	+	Q
T	+	QT
*	+*	QT
V	+*	QTV
/	+*/	QTV
U	+*/	QTVU

INFIX	STACK	PREFIX
/	+*//	QTUV
W	+*//	QTUVW
*	`+* // *`	QTUVW
)	`+*//*)`	QTUVW
P	`+*//*)`	QTUVWP
^	`+* // *)^`	QTUVWP
O	`+* // *)^`	QTUVWPO
(	`+* // *`	QTUVWPO^
+	++	QTUVWPO^ `*//*`
N	++	QTUVWPO^ `*//*` N
*	++*	QTUVWPO^ `*//*` N
M	++*	QTUVWPO^ `*//*` NM
-	++-	QTUVWPO^ `*//*` NM*
L	++-	QTUVWPO^ `*//*` NM*L
+	++-+	QTUVWPO^ `*//*` NM*
K	++-+	QTUVWPO^ `*//*` NM*K
		QTUVWPO^ `*//*` NM*K+-++

Reverse Expression: ++-+KL *MN* // * ^OPWUVTQ

Infix to Postfix

Without Stack

Infix: a + b * c - d / e ^ f

- a + b * c - d / e f^
- a + b c * - d / e f ^
- a + b c

Using Stack

Infix	Stack	Postfix
a		a
+	+	a
b	+	ab
*	+*	ab

Infix	Stack	Postfix
c	+*	abc
-	-	abc*+
d	-	abc*+d
/	-/	abc*+d
e	-/	abc*+de
^	-/^	abc*+de
f	-/^	abc*+def
		abc*+def^/-

Postfix Expression Evaluation

Postfix expression (Reverse Polish Notation) places the operator after the operands.
Example:
infix: (2 + 3) x 4
Postfix: 2 3 + 4 x

Postfix evaluation is straightforward using a stack because it does not require precedence rules or parentheses.

Algorithm: Evaluating Postfix using Stack

1. Initialize an empty stack
2. scan the postfix expression from left to right.
   - If the scanned character is an operand, push it onto the stack
   - If the scanned character is an operator:
     - Pop the top two elements from the stack (let them be op2 and op1 in this order)
     - Apply the operator to these operands: result = op1 operator op2.
     - Push the result back into the stack
3.

Example

Evaluate:

Balancing Parentheses

Balancing parentheses is a common problem in computer science, particularly in the fields compiler design and expression evaluation. The goal is to determine whether the parentheses (or brackets) in a given expression are properly matched and nested.

Concept

In a balanced expression:

(3-2-26)

Queue

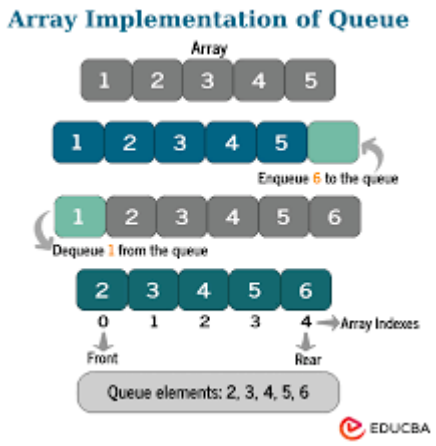
A queue is a linear data structure that follows the FIRST IN, FIRST OUT (FIFO) principle.

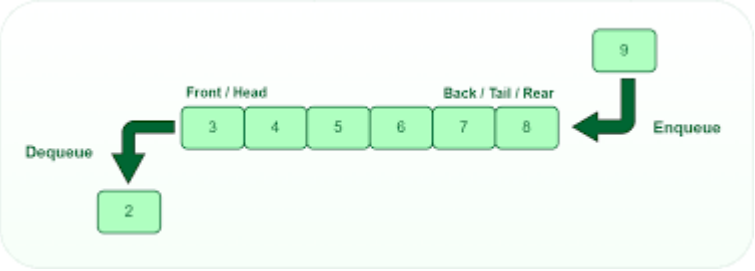
This means that the element inserted first will be removed first. It is analogous to a line of people waiting for a service, where the person at the front is served first.

A queue performs the following set of operations:

1. Enqueue(element): Insert an element at the rear of the queue
2. Dequeue( ): Remove and return the front element from the queue.
3. peek( )/ front ( ): Return the front element without removing it.
4. isEmpty( ): Check whether the queue is empty
5. isFull( ): Check whether the queue is full (in case of fixed size implementation).
6. size( ): Return the number of elements in the queue.

Array Implementation of Queue





Types of Queues

1. Simple Queue: Basic FIFO queue.
2. Circular Queue: Connects the last position back to the first position to utilize free space efficiently.
3. Priority Queue: Each element is assigned s priority, and elements with higher priority are dequeued first.
4. Double-Ended Queue (Deque): Elements can be added or removed from both ends.

Applications of Queues

Queues are widely used in computer science and real-life scenarios:

- CPU Scheduling: Ready queue in operating systems schedules processes in FIFO order
- Print Queue:
- Breadth-first-search:

```
// circular queue code

import java.util.Sacnner;

class CircularQueue{
    private int[] queue;
    private int front;
    private int rear;
    private int capacity;
    private int count;

    public CircularQueue(int size){
        capacity = size;
        queue = new int[capacity];
        front = 0;
        rear = -1;
        count = 0;
    }

    public boolean isFull() {
        return count == capacity;
    }

    public boolean isEmpty(){
        return count == 0
    }

    public void enqueue(int item){
        if (isFull()) {
            System.out.println("Queue is full. Cannot enqueue" + item);
```

```
            return;
        }
        rear = (rear + 1) % capacity;
        queue[rear] = item;
        count++;
        System.out.println(item + " enqueued.");
    }
    public void dequeue() {
        if(isEmpty()) {
            System.out.println("Queue is empty. Cannot dequeue.");
            return;
        }
        int item = queue[front];
        front = (front + 1) % capacity;
        count--;
        System.out.println(item +" dequeued");
    }

    public void peek() {
        if (isEmpty()) {
            System.out.println("Queue is empty");
            return;
        }
        System.out.println("Front element:  "+ queue[front]);
    }

    public void display() {
        if(isEmpty()){
            System.out.println("Queue is Empty.");
            return;
        }
        System.out.println("Queue Elements: ");
        for (int i = 0; i < count; i++){
            int index = (front + i) % capacity;
            System.out.println(queue[index] + " ");
        }
        System.out.println();
    }

}

public class CircularQueueUsingArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter Queue Capacity: ");
        int capacity = sc.nextInt();

        Queue queue = new Queue(capacity);
        int choice;
```

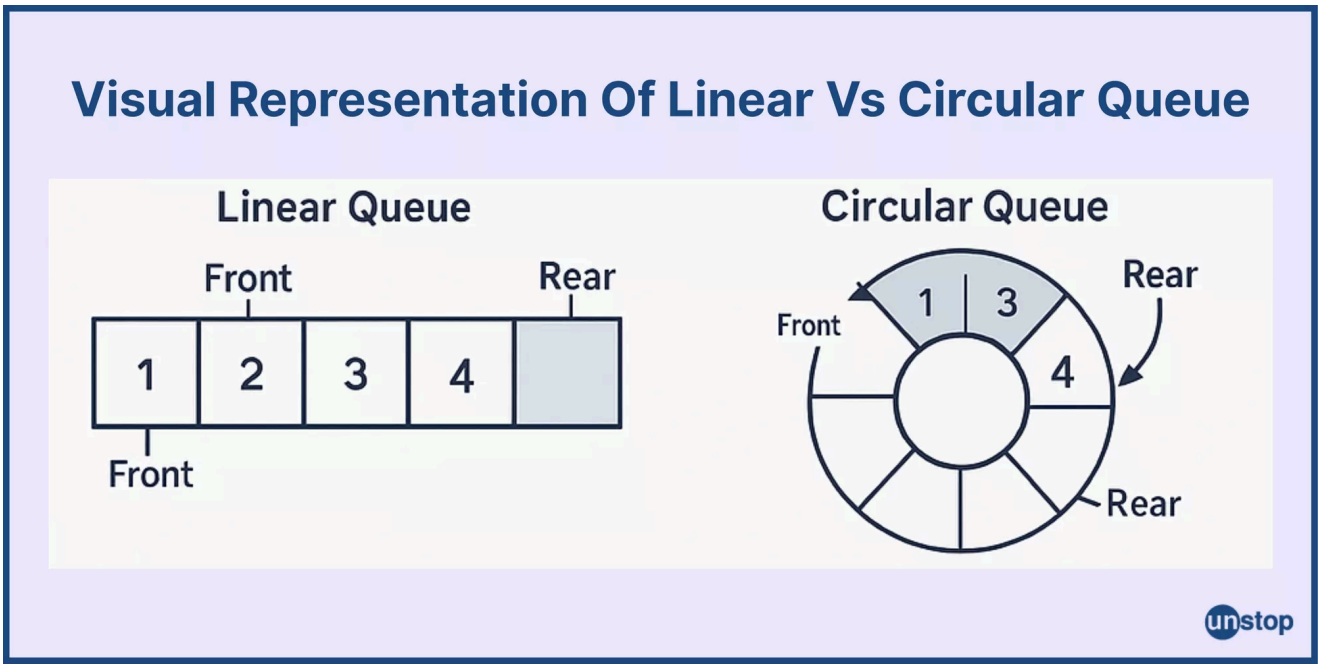
```
        do{
            System.out.println("\nQueue Operations Menu: ");
            System.out.println("1. Enqueue");
            System.out.println("2. Dequeue");
            System.out.println("3. Peek");
            System.out.println("4. Display");
            System.out.println("5. Is Full?");
            System.out.println("6. Is Empty?");
            System.out.println("7. Exit");
            System.out.println("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice){
                case 1:
                    System.out.println("Enter element to enqueue: ");
                    int element = sc.nextInt();
                    queue.enqueue(element);
                    break;
                case 2:
                    queue.dequeue();
                    break;
                case 3:
                    queue.peek();
                    break;
                case 4:
                    queue.display():
                    break;
                case 5:

            }
        }
    }
}
```

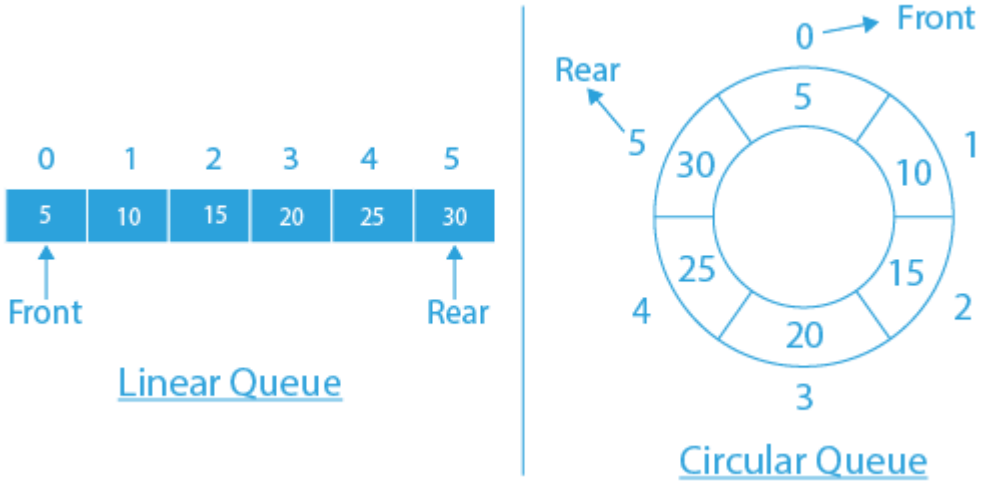
Circular Queue

A circular Queue is a linear data structure that overcomes the limitation of the simple queue's wasted space by connecting the end of the queue back to the front, forming a circle. This allows efficient utilization of the storage by reusing the empty spaces created by dequeuing elements.

Characteristics

1. Follows First In First Out (FIFO) principle.
2. Rear and front pointers wrap around when they reach the end of the array.
3. Helps to avoid the "queue full" problem seen in simple queues where space is wasted.





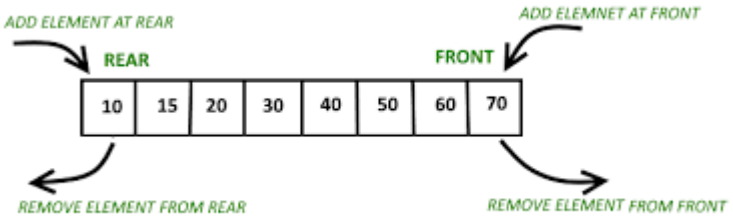

```
import java.util.Sacnner;

class CircularQueue{
    private int[] queue;
    private int front;
    private int rear;
    private int capacity;
    private int count;

    public CircularQueue(int size){
        capacity = size;
        queue = new int[capacity];
        front = 0;
        rear = -1;
        count = 0;
    }

    public boolean isFull() {
        return count == capacity;
```

```
    }

    public boolean isEmpty(){
        return count == 0
    }

    public void enqueue(int item){
        if (isFull()) {
            System.out.println("Queue is full. Cannot enqueue" + item);
            return;
        }
        rear = (rear + 1) % capacity;
        queue[rear] = item;
        count++;
        System.out.println(item + " enqueued.");
    }
    public void dequeue() {
        if(isEmpty()) {
            System.out.println("Queue is empty. Cannot dequeue.");
            return;
        }
        int item = queue[front];
        front = (front + 1) % capacity;
        count--;
        System.out.println(item +" dequeued");
    }

    public void peek() {
        if (isEmpty()) {
            System.out.println("Queue is empty");
            return;
        }
        System.out.println("Front element:  "+ queue[front]);
    }

    public void display() {
        if(isEmpty()){
            System.out.println("Queue is Empty.");
            return;
        }
        System.out.println("Queue Elements: ");
        for (int i = 0; i < count; i++){
            int index = (front + i) % capacity;
            System.out.println(queue[index] + " ");
        }
        System.out.println();
    }

}
```

```
public class CircularQueueUsingArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter Queue Capacity: ");
        int capacity = sc.nextInt();

        Queue queue = new Queue(capacity);
        int choice;
        do{
            System.out.println("\nQueue Operations Menu: ");
            System.out.println("1. Enqueue");
            System.out.println("2. Dequeue");
            System.out.println("3. Peek");
            System.out.println("4. Display");
            System.out.println("5. Is Full?");
            System.out.println("6. Is Empty?");
            System.out.println("7. Exit");
            System.out.println("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice){
                case 1:
                    System.out.println("Enter element to enqueue: ");
                    int element = sc.nextInt();
                    queue.enqueue(element);
                    break;
                case 2:
                    queue.dequeue();
                    break;
                case 3:
                    queue.peek();
                    break;
                case 4:
                    queue.display():
                    break;
                case 5:

            }
        }
    }
}
```

Double-Ended Queue

A double-ended queue is a generalized version f a queue where elements can be inserted or removed from both the front and rear ends. It combines the properties of both stacks and queues.



Operations

- insertFront(element):
- insertRear(element):
- deleteFront():
- deleteRear():
- isFull():
- isEmpty():
- peekFront():
- peekRear():